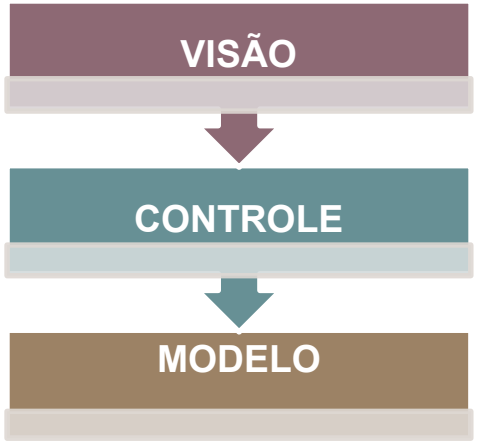
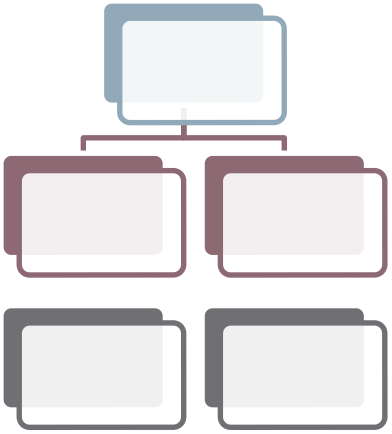
Arquitetura e Desenho de Software

AULA - MODELAGEM UML ESTÁTICA



Profa. Milene Serrano



UnB Gama

Agenda

Considerações Iniciais

Abordagens Tradicionais

- Modelagem Orientada a Objetos
  - UML (*Unified Modeling Language*)
  - Diagramas Estáticos e Organizacionais

Considerações Finais

- Conheceremos em tempo de aula, principalmente, os diagramas de classe, pacote e componente.
- Mas, existem outros diagramas ...
- Qualquer dúvida, por gentileza, entrar em contato.

Considerações Iniciais

Considerações Iniciais

Existem várias propostas de abordagens anteriores à Orientação a Objetos.

Mas, nesse curso, iremos focar em modelos comumente utilizados no desenho de soluções orientadas pelo Paradigma Orientado a Objetos.

Nesses cenários, destaca-se a UML (***Unified Modeling Language***). Trata-se de uma linguagem (uma notação) de modelagem muito reconhecida e utilizada na comunidade de software.

A UML foi criada visando estabelecer uma linguagem visual comum no complexo mundo do desenvolvimento de software, que também poderia ser compreendida por usuários das áreas de negócio e demais envolvidos na elaboração de produtos de software.

Considerações Iniciais

A UML não é, portanto:

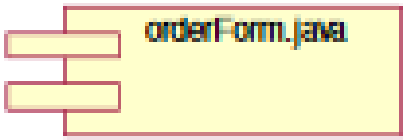
- um processo;
- uma linguagem de programação;
- uma metodologia, ou
- algo nesse sentido.

É, na verdade, uma notação visual, com diagramas que ajudam a desenhar uma solução computacional em diferentes níveis de abstração e/ou perspectivas...

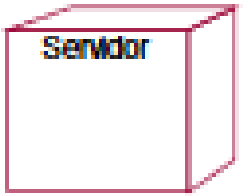
Muitos autores apresentam a UML, em uma visão mais otimizada, como um conjunto de diagramas para detalhamento estrutural (ou estático), dinâmico (ou comportamental); organizacional (ou em pacotes), e anotacional (ou explicativo, com uso de notas) no intuito de modelar uma solução computacional.

Considerações Iniciais

Classe



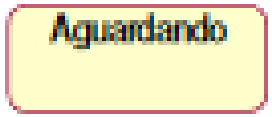
Componente



Nó

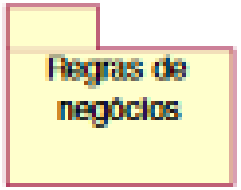
Estruturais ou Estáticos

Mensagem



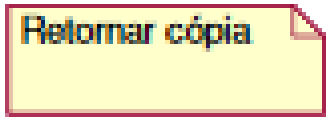
Estado

Comportamentais ou Dinâmicos



Pacote

Organizacionais

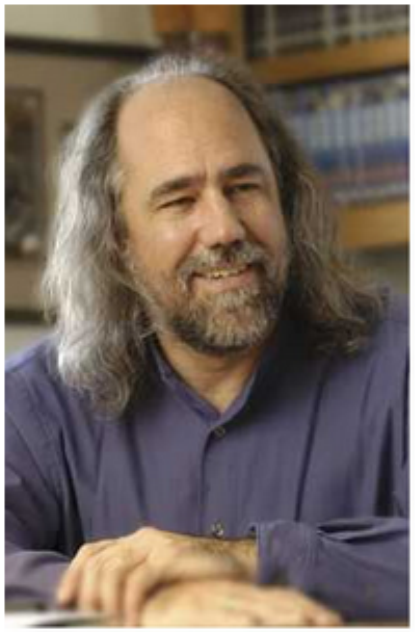


Nota

Anotacionais

Como são muitos os diagramas oferecidos pela UML, iremos focar apenas em alguns, sendo esses os mais utilizados em modelagens estáticas, organizacionais e dinâmicas de soluções orientadas a objetos e associadas.

Considerações Iniciais



grady booch

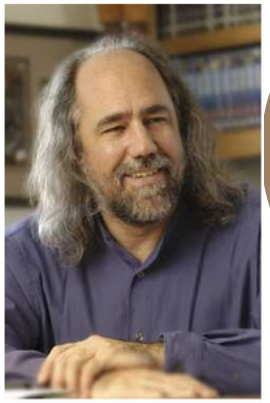
ivar jacobson

james rumbaugh

Vale ressaltar que a UML, em sua origem, procurou reunir as "melhores práticas de engenharia" que provaram ter sucesso na modelagem de sistemas grandes e complexos, na época.

Portanto, está baseada nos conceitos de Booch, OMT (Rumbaugh) e OOSE (Jacobson), fundindo-os em uma única linguagem de modelagem, comum e largamente utilizada.

Considerações Iniciais



grady booch

BOOCH

- Diagrama de Estados
- Diagrama de Colaboração
- Outros

ivar jacobson

OOSE

- Diagrama de Casos de Uso
- Diagrama de Pacotes
- Outros

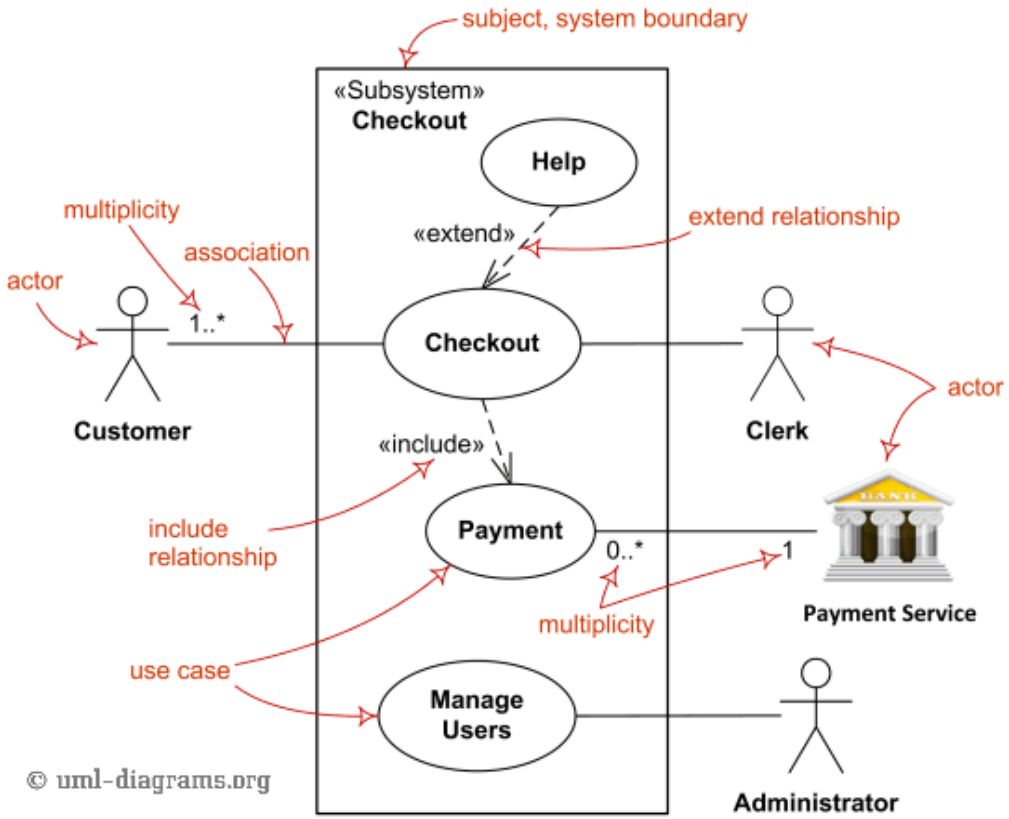
UML

james rumbaugh

OMT

- Diagrama de Estados
- Diagrama de Classe

Considerações Iniciais



Vocês já conhecem bem um diagrama dinâmico da UML, o qual revela uma parte comportamental da solução computacional. Trata-se do Diagrama de Casos de Uso! Lembram-se dele? :)

Diagrama de Classe

Diagrama de Classe

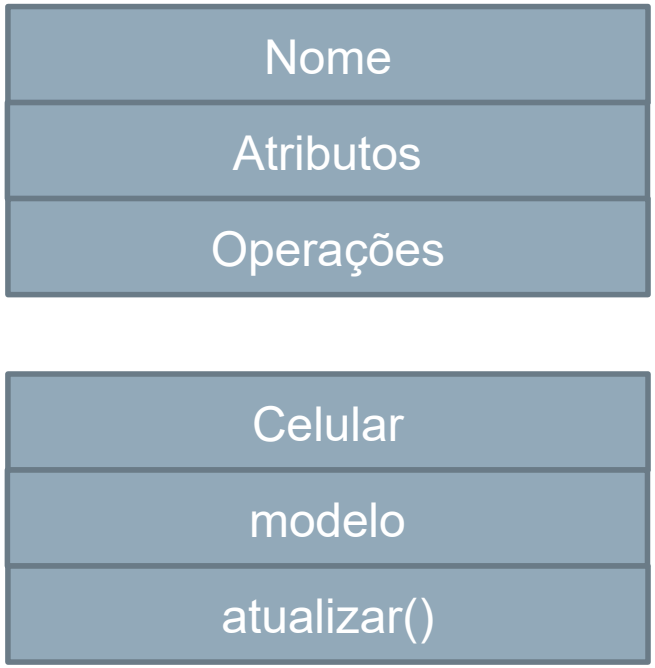
É um diagrama estático da UML que reúne os elementos mais importantes de um sistema orientado a objetos.

Nele são exibidos um conjunto de classes, interfaces e seus relacionamentos.

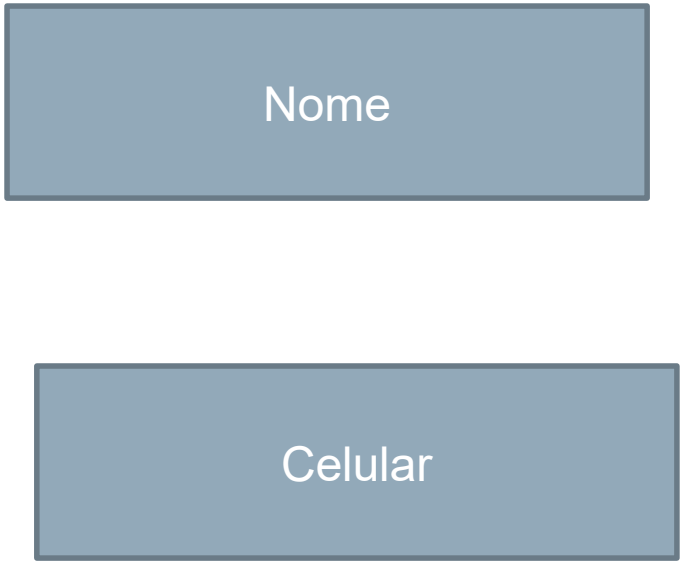
As classes especificam tanto as propriedades quanto os comportamentos dos objetos.

https://www.uml-diagrams.org/class-diagrams-overview.html

Diagrama de Classe

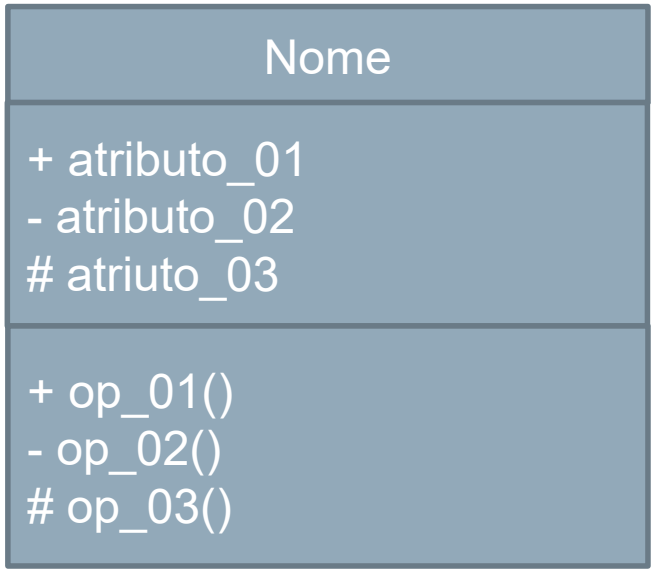


OU



Existem Outras Alternativas...
Itálico, normalmente, representa **<<*abstract*>>**

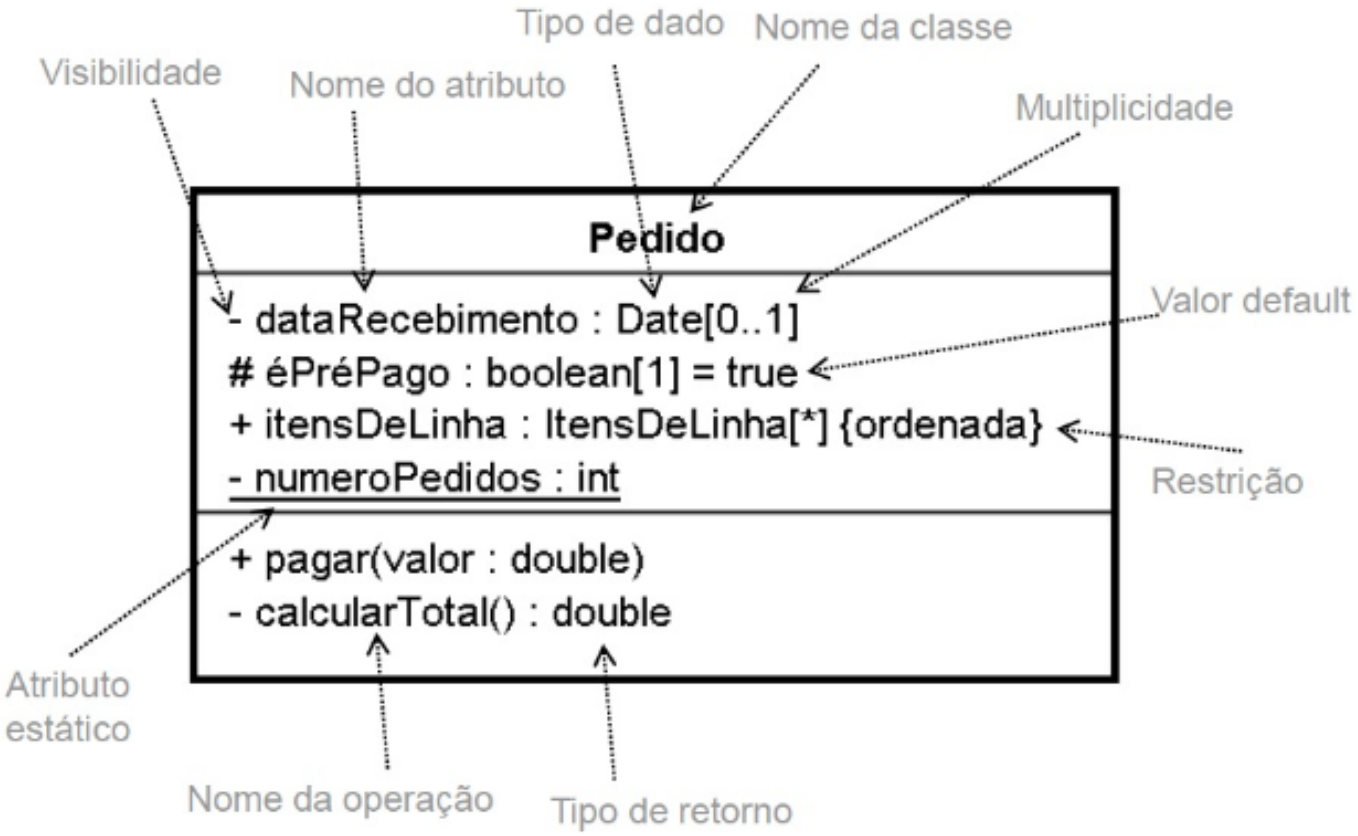
Diagrama de Classe



+	public
-	private
#	protected

Especificações de Acesso
Visibilidade
Escopo

Diagrama de Classe



Estrutura de uma Classe
Algumas particularidades

Dependendo da ferramenta de modelagem, essas especificações podem variar um pouco...

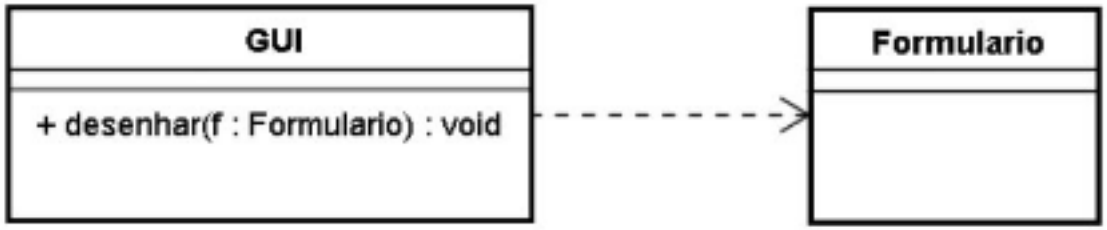
Diagrama de Classe

Relacionamentos:

- Dependência
- Associação
- Agregação
- Composição
- Generalização
- Realização

DEPENDÊNCIA

Diagrama de Classe

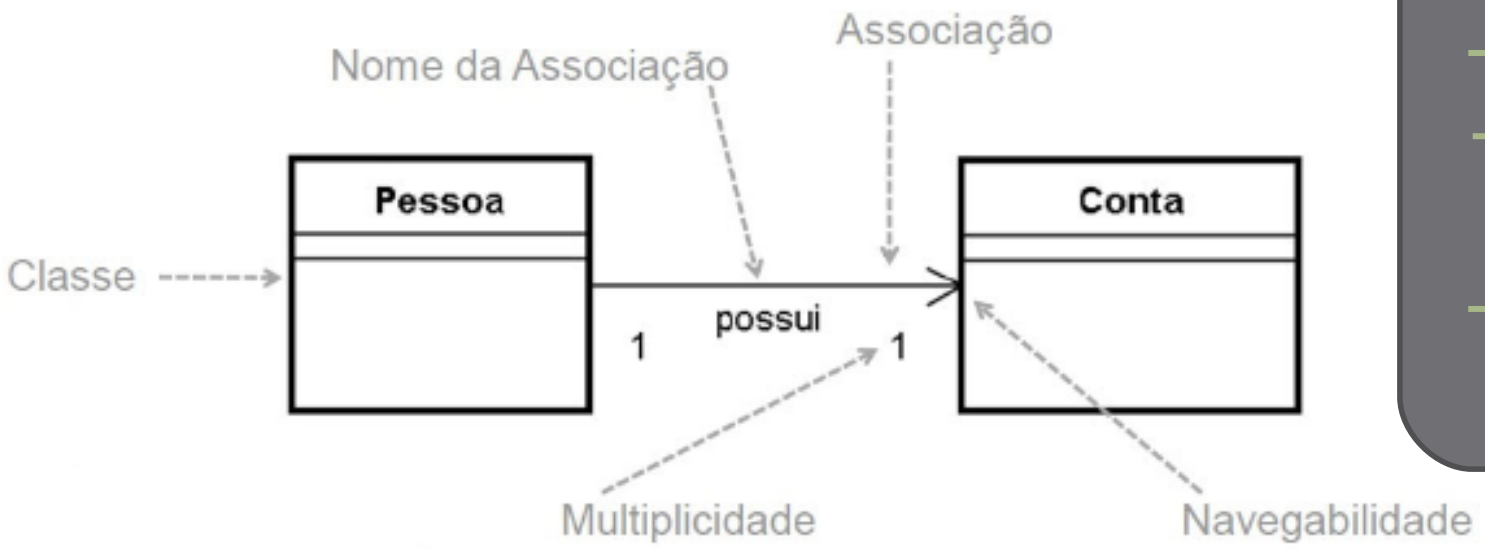



```
public class GUI {

        public void desenhar(Formulario f) {
                f.pintarBotao();
                f.pintarMenu();
                f.pintarJanelas();
                ...
        }
}
```

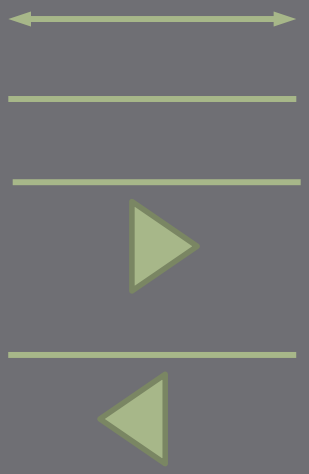
Dependência

ASSOCIAÇÃO

Diagrama de Classe



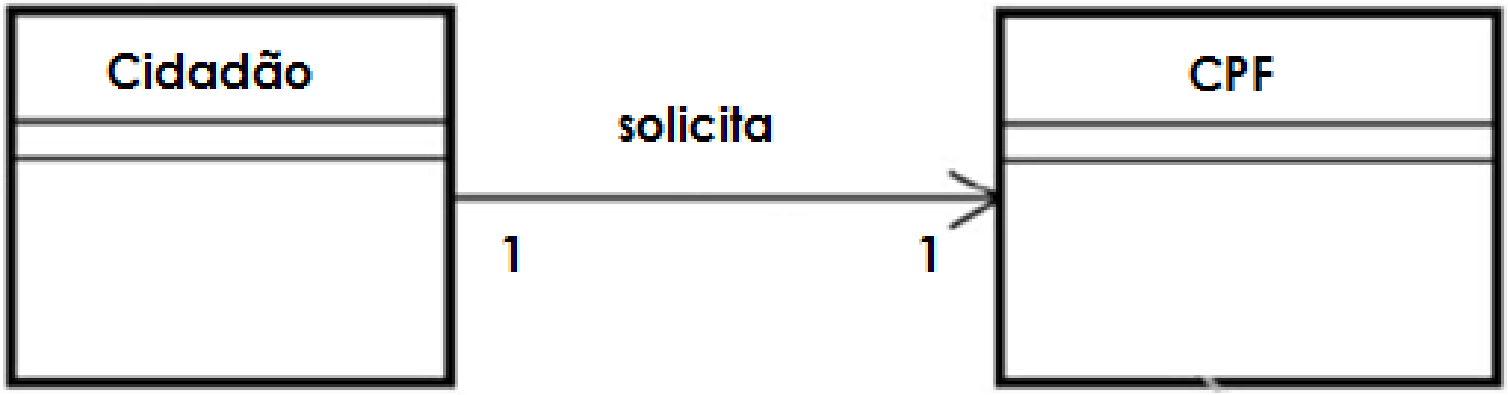
Outras representações:



Associação

- PESSOA POSSUI CONTA (voz ativa, sentido da navegabilidade)
- CONTA É_POSSUÍDA PELA PESSOA (voz passiva, navegabilidade oposta)

Diagrama de Classe

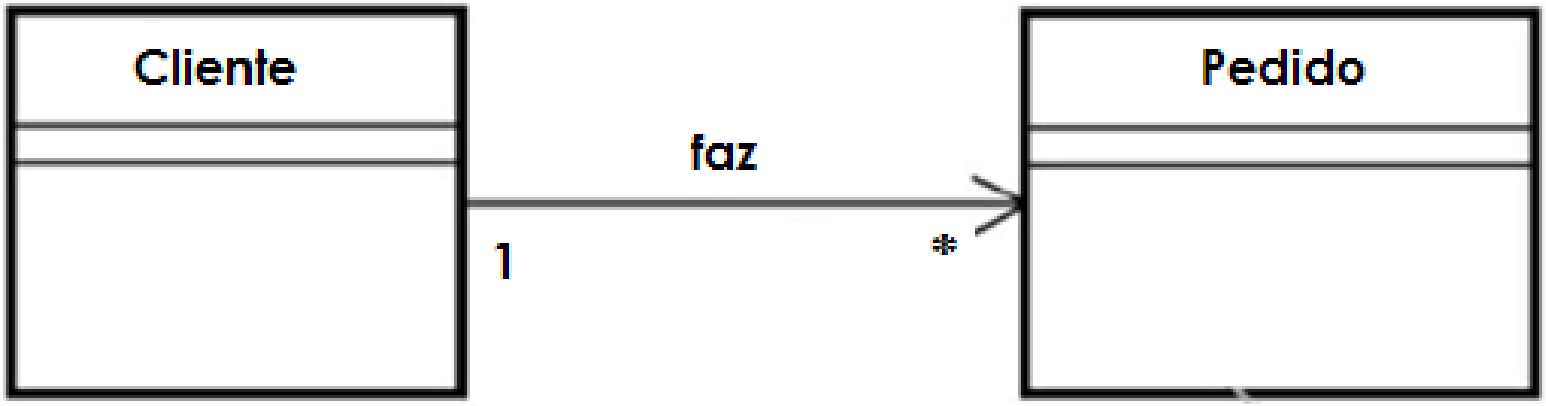


UM CIDADÃO PODE SOLICITAR APENAS UM CPF (1 .. 1)
O MESMO CPF SÓ PODE SER SOLICITADO POR UM CIDADÃO (1 .. 1)
RESULTANTE: 1.. 1

Associação 1..1 (UM PARA UM)

- CIDADÃO SOLICITA CPF (voz ativa, sentido da navegabilidade)
- CPF É_SOLICITADO PELO CIDADÃO (voz passiva, navegabilidade oposta)

Diagrama de Classe

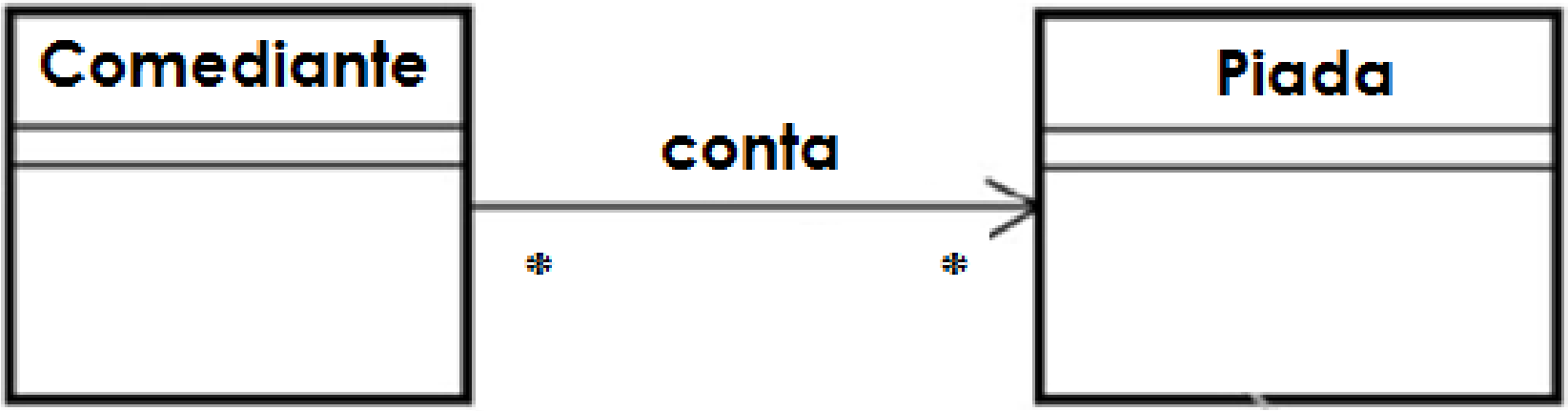


UM CLIENTE PODE FAZER VÁRIOS PEDIDOS (1 .. *)
O MESMO PEDIDO SÓ PODE SER FEITO POR UM CLIENTE (1 .. 1)
RESULTANTE: 1 .. *

Associação 1..N ou 1..* (UM PARA MUITOS)

- CLIENTE FAZ PEDIDO (voz ativa)
- PEDIDO É_FEITO PELO CLIENTE (voz passiva)

Diagrama de Classe



UM COMEDIANTE CONTA VÁRIAS PIADAS (1 .. *)
A MESMA PIADA PODE SER CONTADA POR VÁRIOS COMEDIANTES (1 .. *)
RESULTANTE: * .. *

Associação N .. N ou * .. * (MUITOS PARA MUITOS)

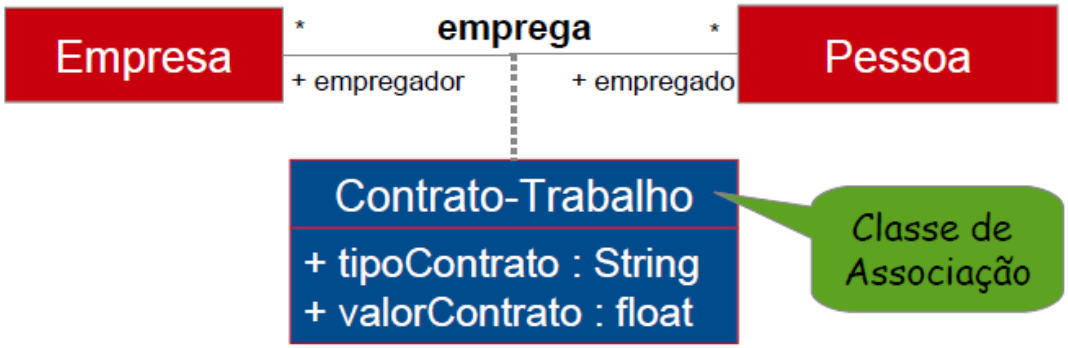
- COMEDIANTE CONTA PIADA (voz ativa)
- PIADA É_CONTADA PELO COMEDIANTE (voz passiva)

Diagrama de Classe

Existem outras cardinalidades ou multiplicidades....

- 1 .. 2
- 1 .. [0 .. 1]
- 1 .. [2 .. 5]
- e outras combinações...

Diagrama de Classe

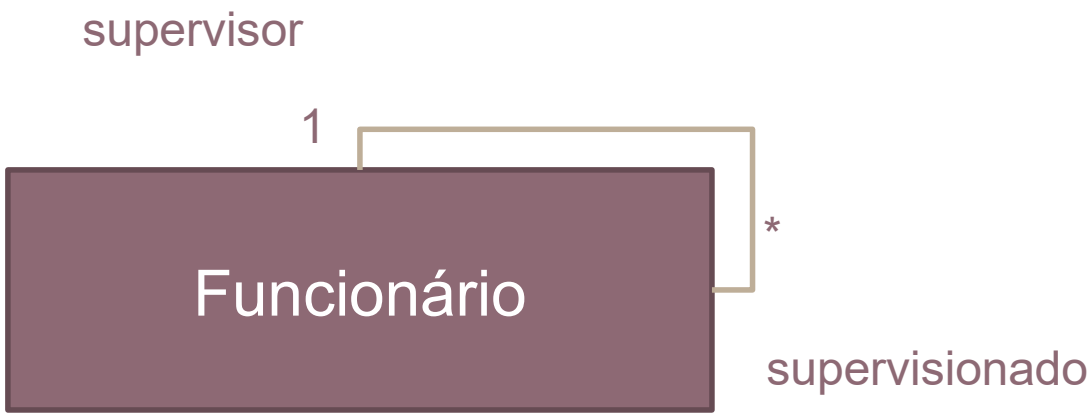


Imaginem um modelo de negócios bem aberto, com empresas contratando.
Política de contratos não exclusivos.
Do relacionamento muitos para muitos, é gera uma tabela associativa, no nível do banco de dados.
No nível de diagrama de classes, essa semântica é modelada usando o conceito de Classe de Associação, conforme consta no exemplo.
Trata-se de uma classe que "nasce" do relacionamento.

Classe Associativa ou Classe de Associação

- Em relacionamentos entre classes, em especial os de cardinalidade *..*, pode-se ter a necessidade de manter informações sobre esta associação. Sendo assim, modela-se uma classe associativa, conforme ilustrado no exemplo.

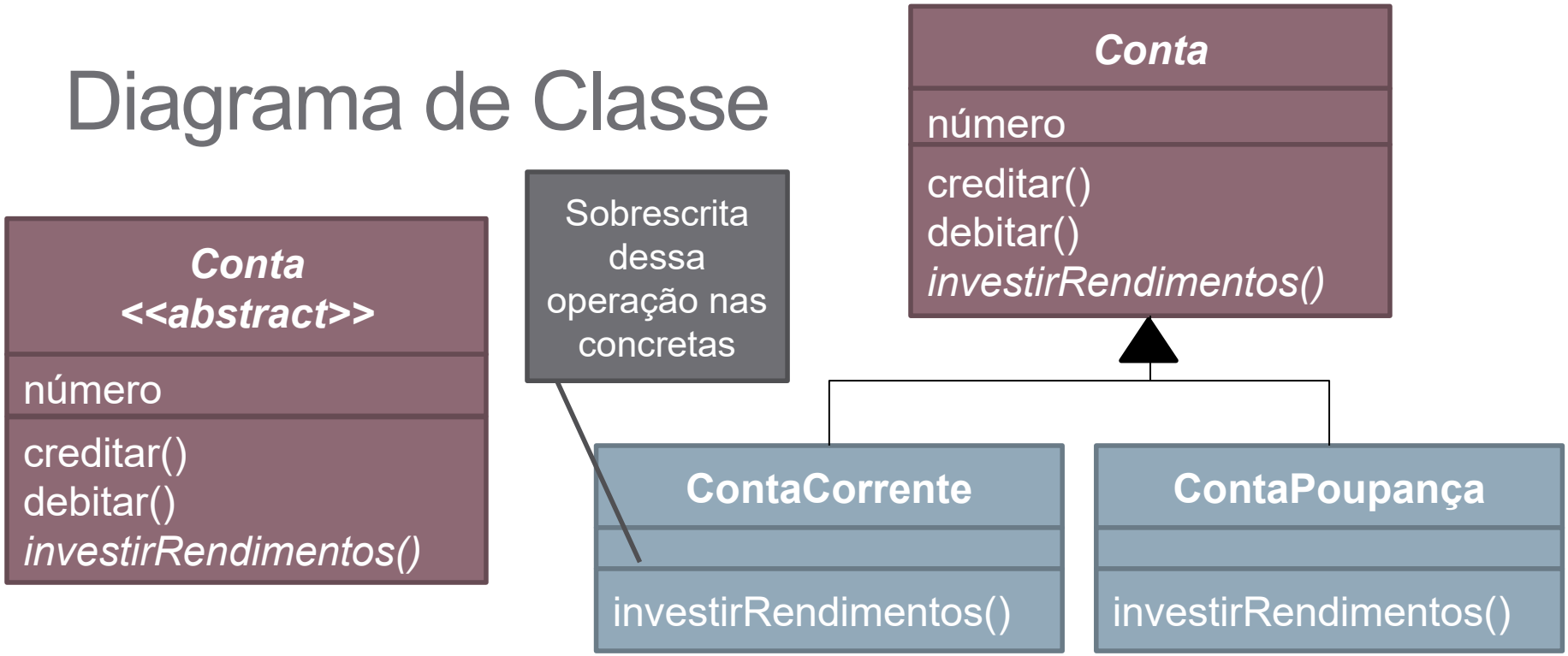
Diagrama de Classe



Associação Reflexiva

- Associação que representa ligações entre objetos que pertencem a uma mesma classe.
- A definição de papéis é importante para evitar ambiguidades na leitura da associação.
  - Cada objeto tem um papel distinto na associação.
    - Ex: “supervisor ” e “supervisionado”.

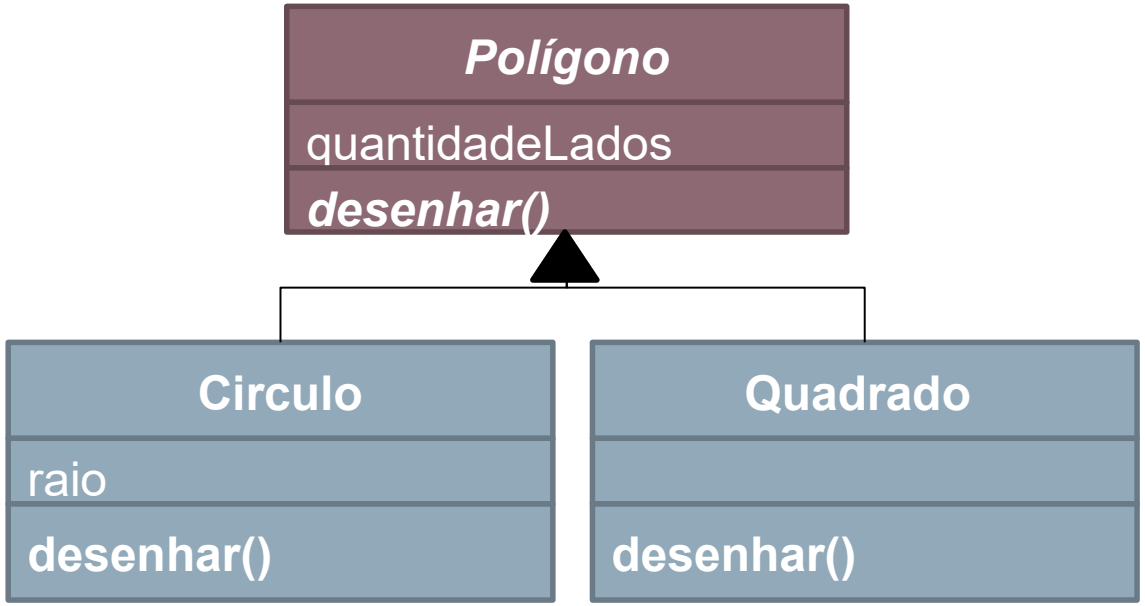
Diagrama de Classe



Classe Concreta e Classe Abstrata

- Normalmente, a existência de uma classe se justifica pelo fato de haver a possibilidade de gerar instâncias da mesma
  - Essas são as ***classes concretas.***
- No entanto, podem existir classes que não geram instâncias diretas.
  - Essas são as ***classes abstratas.***

Diagrama de Classe



Princípio de OO

Sobrescrita - Polimorfismo

- Mesma assinatura, comportamentos diferentes...

Diagrama de Classe

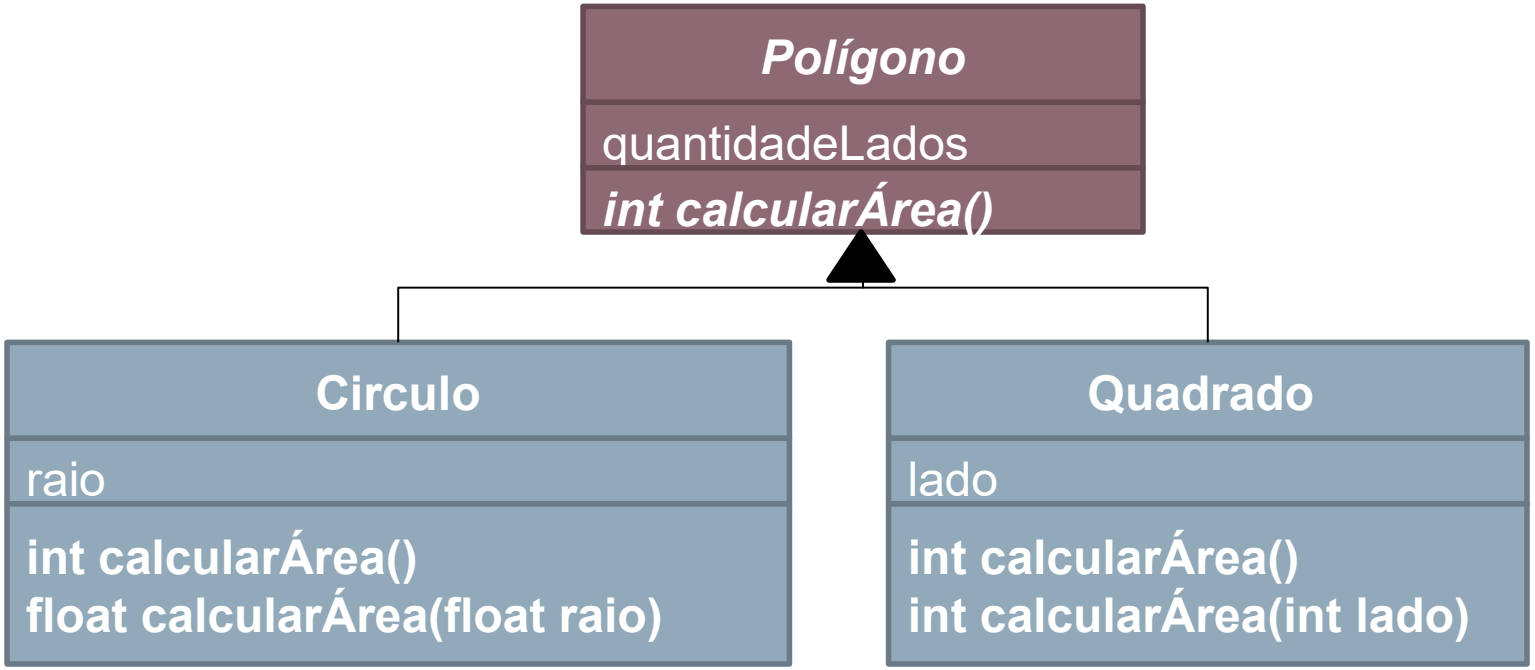
Princípio de OO

Aluno
nome númeroMatrícula
cadastrarAluno(String nome) cadastrarAluno(String nome, int númeroMatrícula)

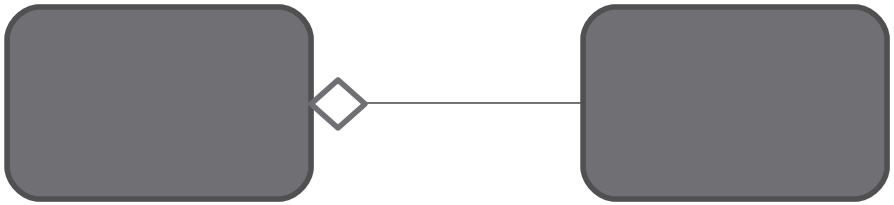
Sobrecarga

- Mesmo nome, mas com parâmetros e/ou retorno diferentes..
- Comumente utilizada em construtores...
- Mas, pode ser aplicada em qualquer operação...

Diagrama de Classe



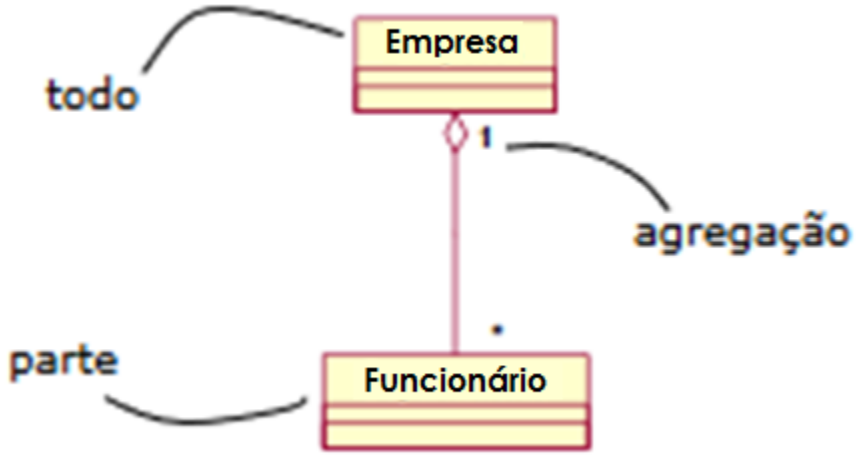
Não confundir sobrecarga e sobrescrita...

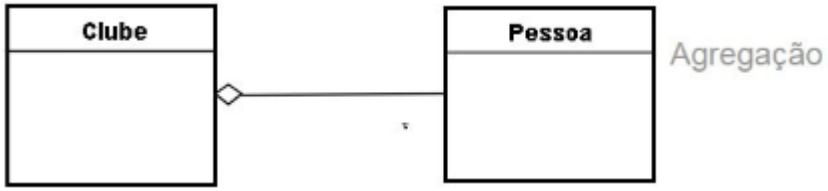


AGREGAÇÃO

Diagrama de Classe

A semântica mais próxima para esse relacionamento é "tem"





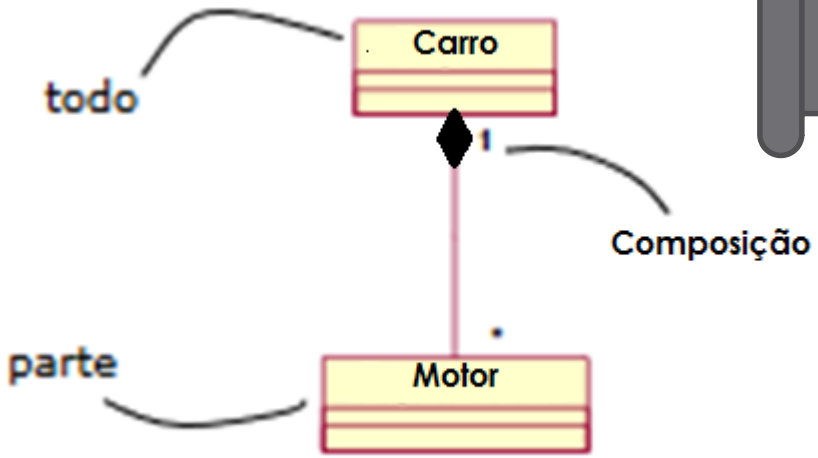
Agregação

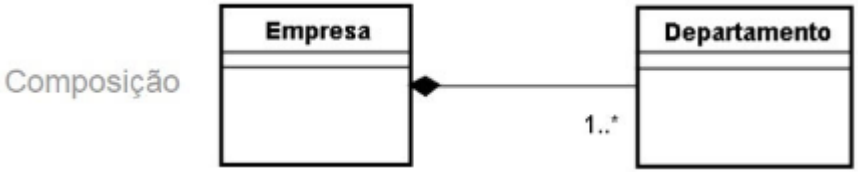
- A EMPRESA EXISTE, INDEPENDENTEMENTE DE EXISTIR UM OU MAIS FUNCIONÁRIOS, nesse caso...

COMPOSIÇÃO

Diagrama de Classe

As semânticas mais próximas para esse relacionamento são “contém” ou “é composto(a) de”





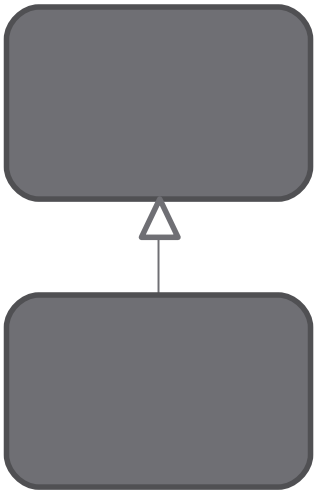
Composição

- NÃO EXISTE CARRO SEM MOTOR, nesse caso...

Diagrama de Classe

Tanto agregação, quando composição podem ter diferentes cardinalidades...

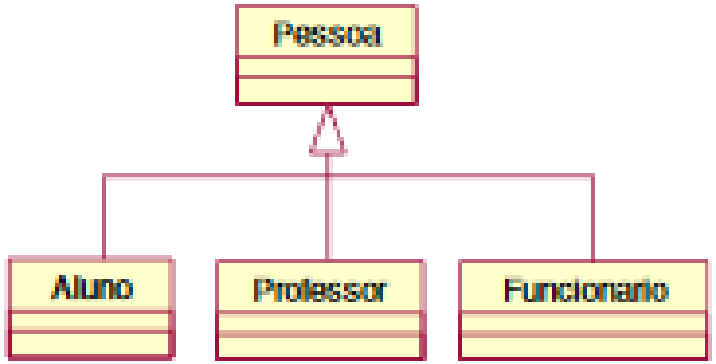
- 1 .. 1
- 1 .. *
- e outros



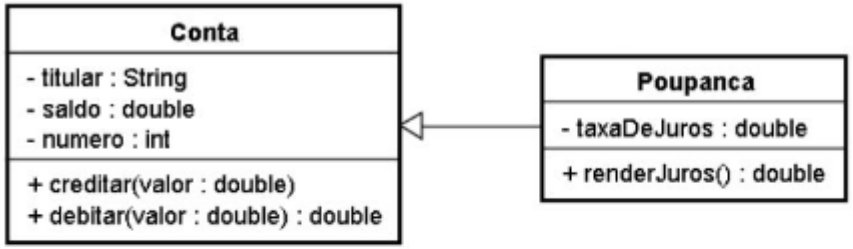
GENERALIZAÇÃO (HERANÇA)

Diagrama de Classe

A semântica mais próxima para esse relacionamento é “é_um(a)”



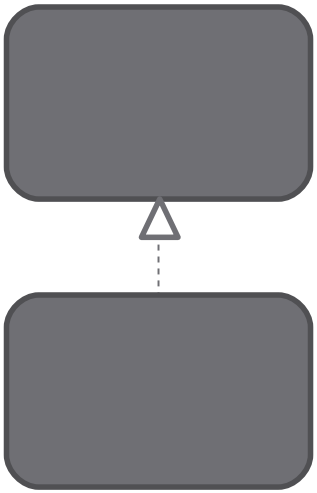
<<EXTENDS>>




```
public class Conta {
        private String titular;
        private double saldo;
        private int numero;
        public void creditar(double valor) {...}
        public double debitar (double valor) {...}
}

public class Poupanca extends Conta {
        private double taxaDeJuros;
        public double renderJuros () {...}
}
```

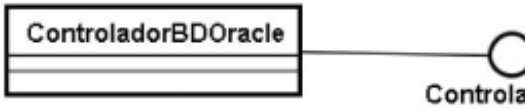
Generalização

- ALUNO É_UMA PESSOA
- PROFESSOR É_UMA PESSOA
- FUNCIONÁRIO É_UMA PESSOA



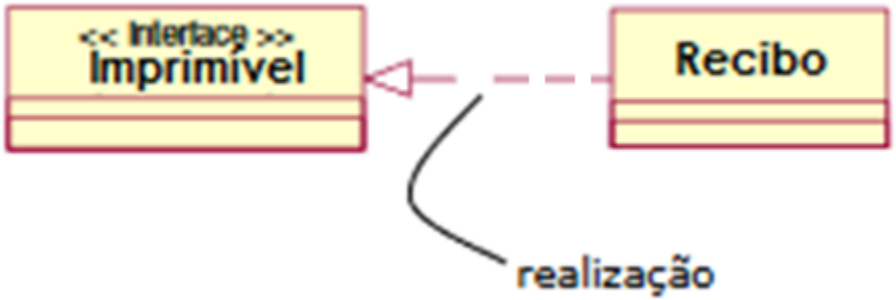
REALIZAÇÃO (INTERFACE)

Diagrama de Classe




```
public interface ControladorBD {

        public void insert(String SQL);
        public Object select(String SQL);
        ...
}
```

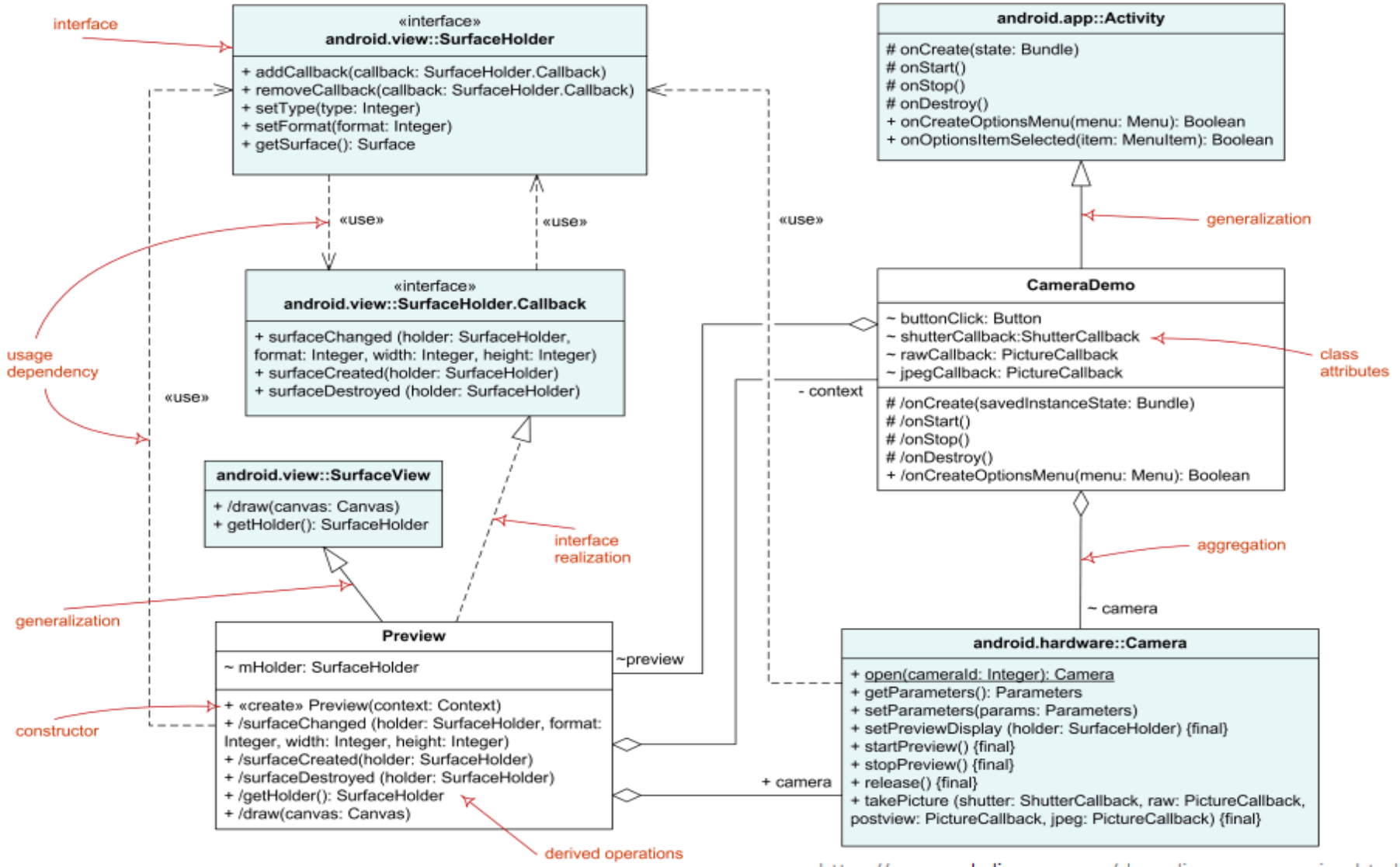
```
public class ControladorOracle
        implements ControladorBD {

        public void insert(String SQL) {
                ...
        }
        public Object select(String SQL) {
                ...
        }
}
```



<<IMPLEMENTS>>

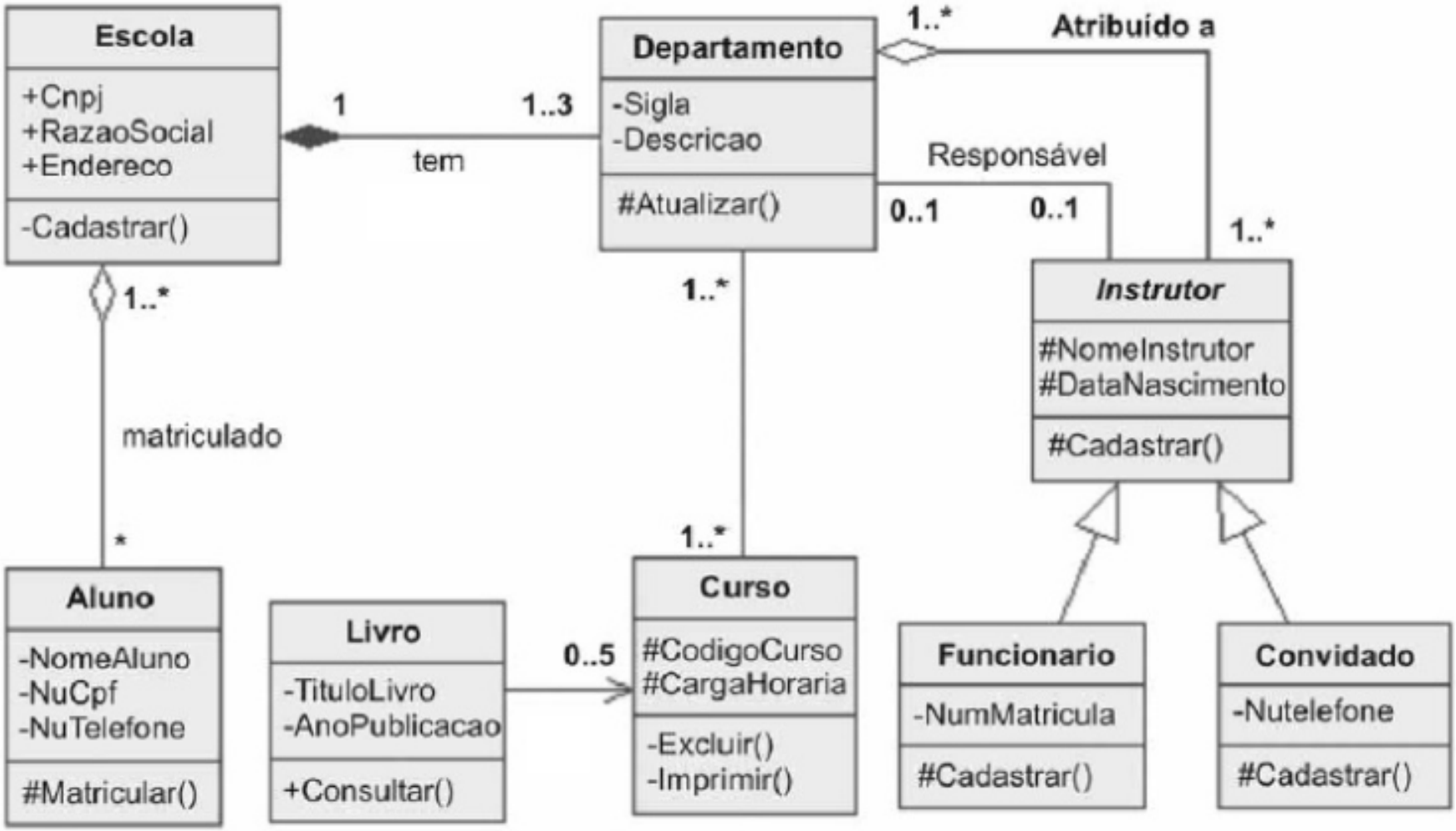
Realização

Diagrama de Classe



https://www.uml-diagrams.org/class-diagrams-overview.html

Diagrama de Classe



www.ricardobarcelar.com.br/aulas/eng_sw/mod3-uml.pdf

Diagrama de Pacotes

Diagrama de Pacotes

Trata-se de mais um diagrama estrutural, estático, o qual permite organizar o sistema como se representasse uma visão em módulos.

https://www.uml-diagrams.org/package-diagrams-overview.html

Diagrama de Pacotes

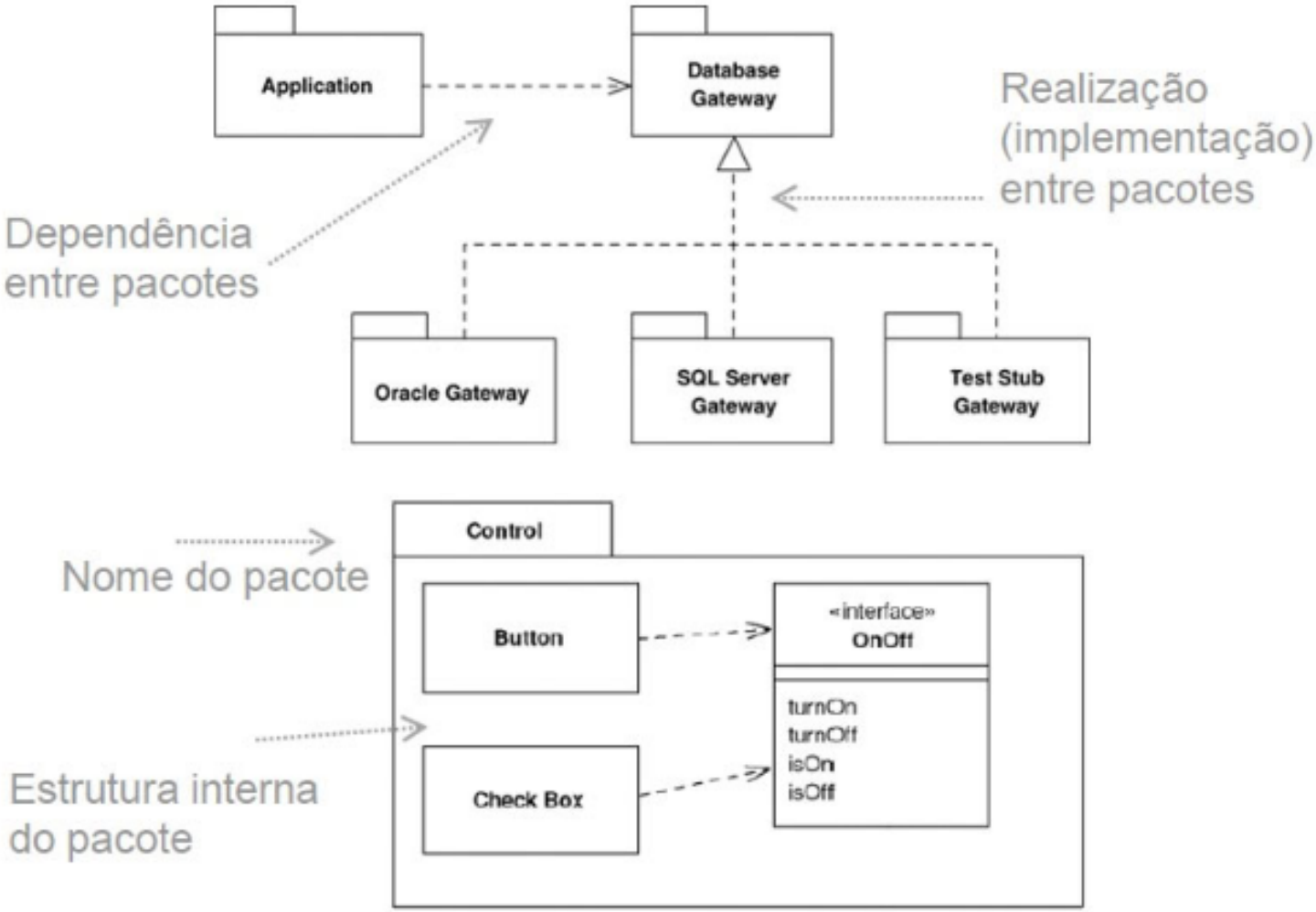


Diagrama de Pacotes

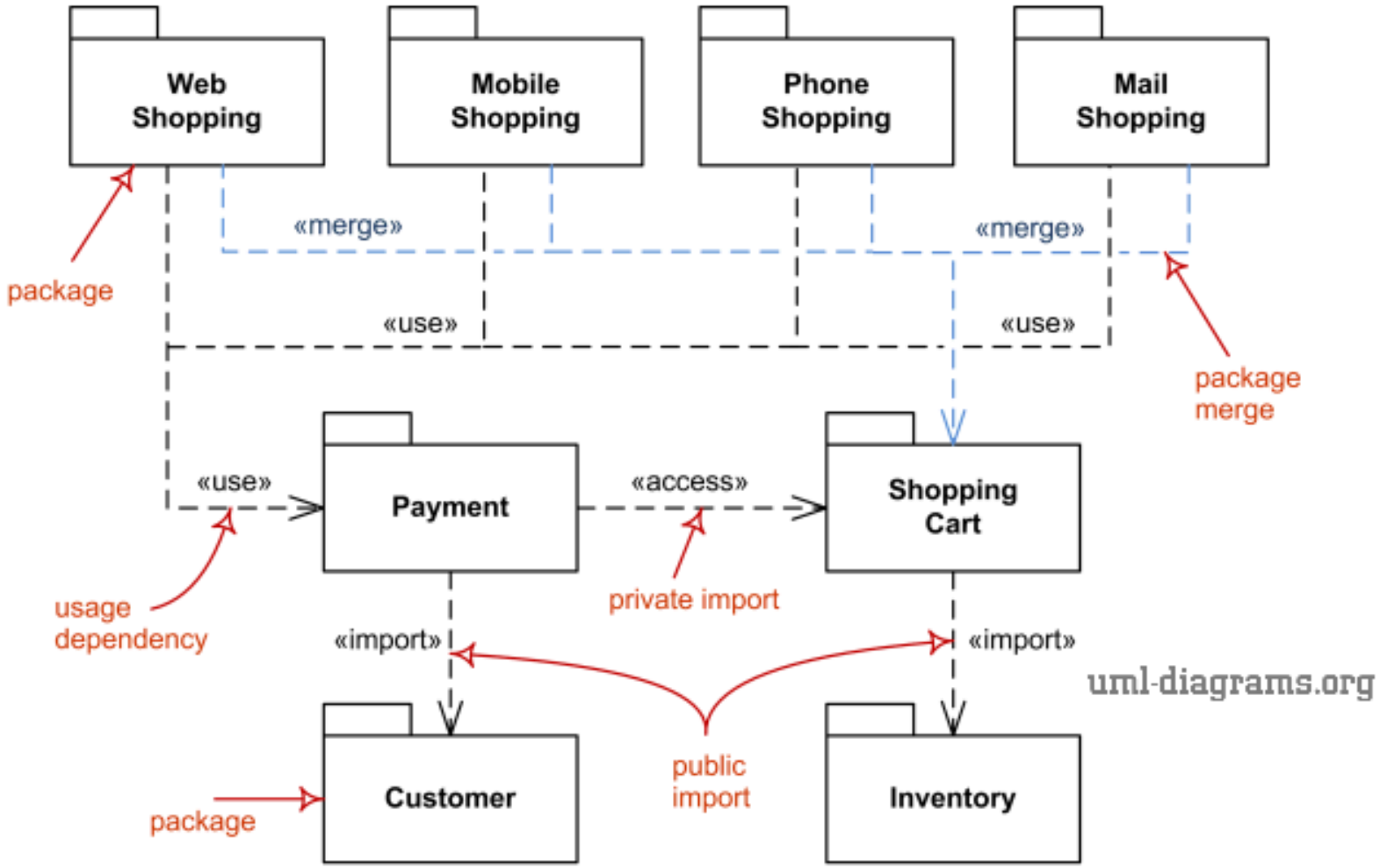


Diagrama de Pacotes

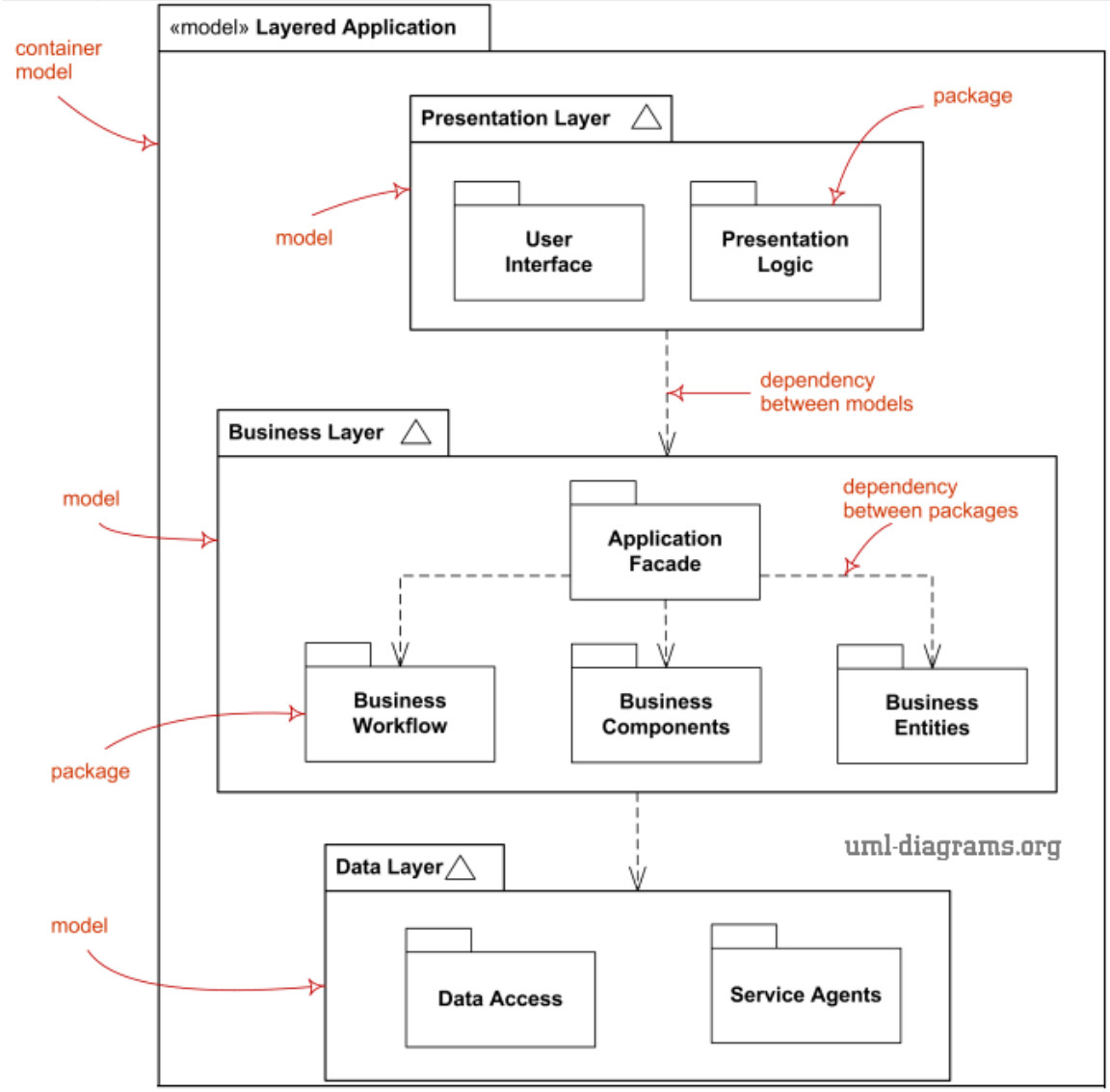


Diagrama de Componentes

Diagrama de Componentes

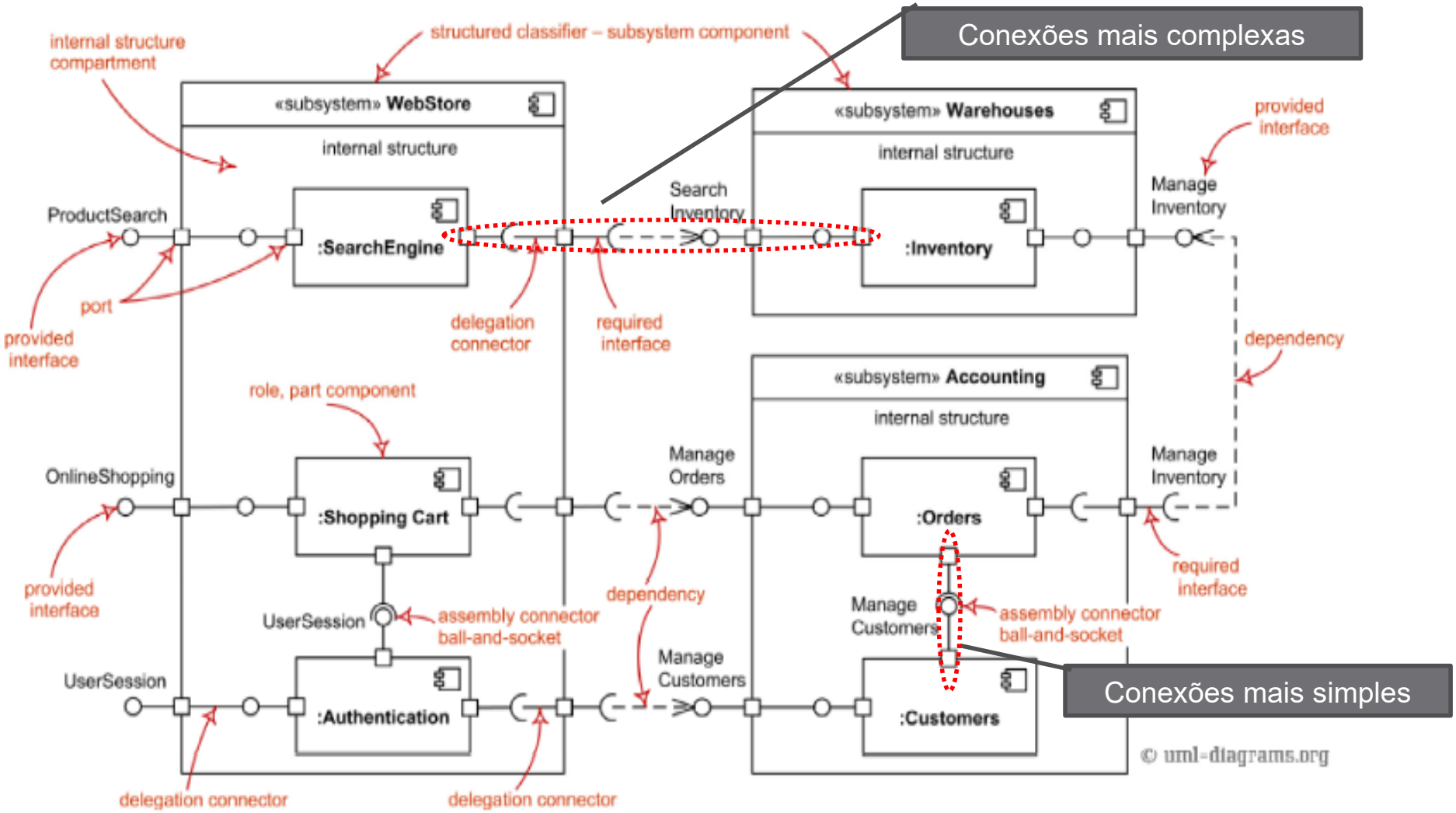
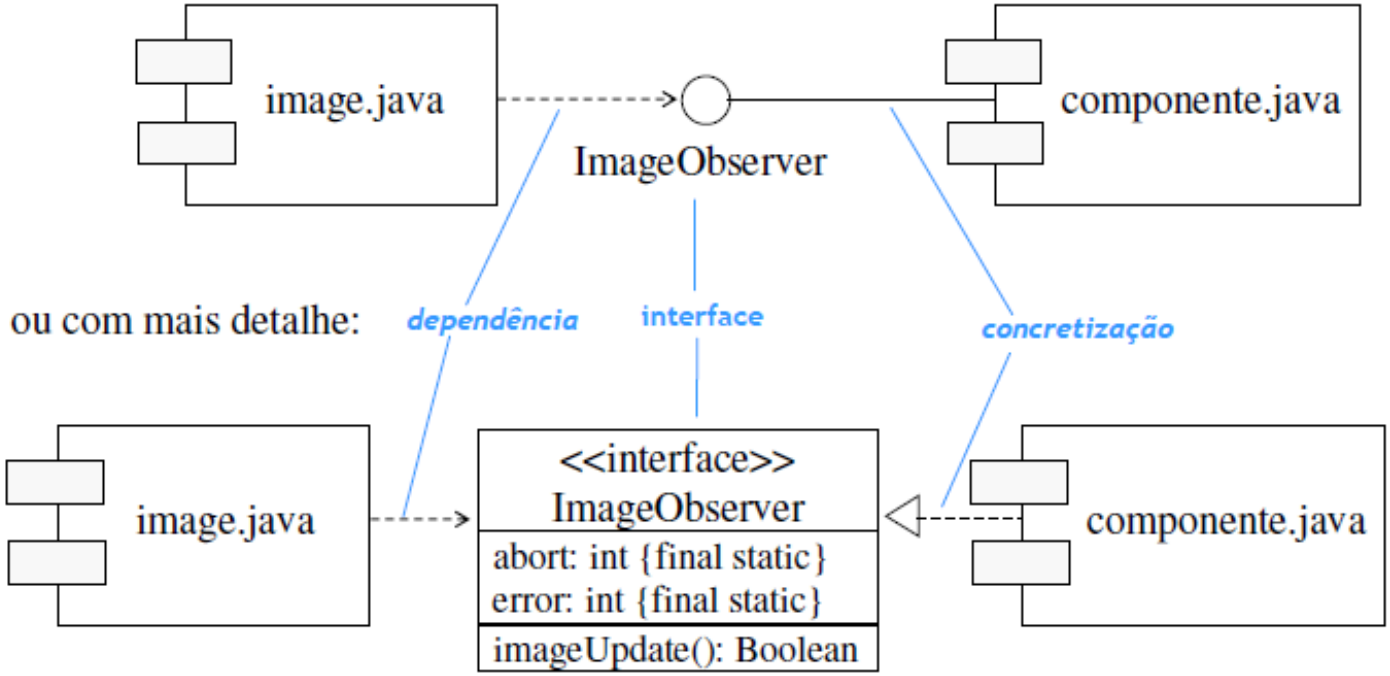


Diagrama de Componentes

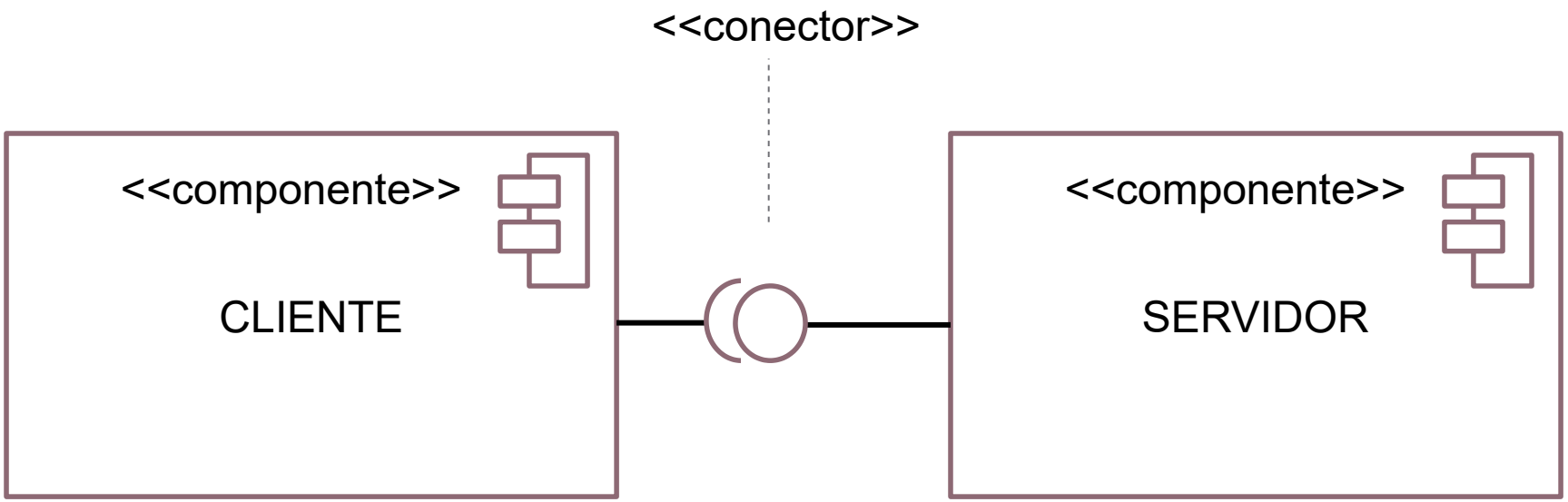


Quem requer VERSUS Quem oferece equivalente a Quem depende VERSUS Quem realiza

FONTE: Crespo, Sérgio. ML – Diagramas de Componentes, Apresentação em PPT, 2010. (disponível na Internet para consulta)

* As notações podem variar, dependo da ferramenta de modelagem utilizada.

Diagrama de Componentes



Desempenha o papel de
“Chamador” ou “Requerente”

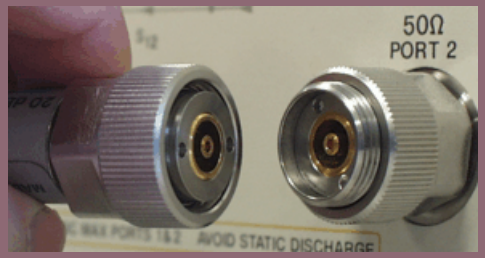
Desempenha o papel de
“Chamado” ou “Requerido”

Típica Arquitetura Cliente-Servidor
Representação com o Diagrama de Componentes

Linguagem de Descrição – Arquitetura Cliente-Servidor

```
System cli_serv = {
   Componente cliente = {
      Port send-request;
      ...
   }
   Componente servidor = {
      Port receive-request;
      ...
   }

   Conector rpc = {
      Roles {chamador, chamado}
      ...
   }

 Configurations = {
      cliente.send-request to rpc.chamador;
      servidor.receive-request to rpc.chamado;      ...
   }
}
```

RPC é a designação de Rosenberger's para o conector APC-7 dele, sendo um ACP-7 um conector coaxial.



Considerações Finais

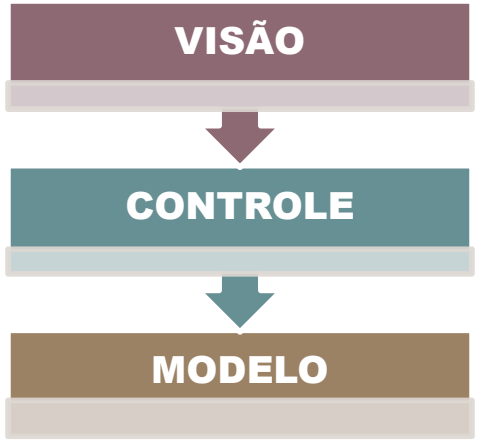
Considerações Finais

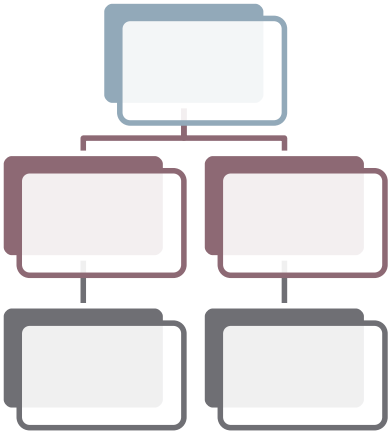
- Nessa aula, apresentamos os principais diagramas estáticos/organizacionais da notação UML.
-
- Existem vários outros artefatos de requisitos e desenho/projeto de software. Acessem:
  - https://www.uml-diagrams.org/
-
- Continuem os estudos!

Referências

Referências

- UML:
  - https://www.uml-diagrams.org/
  - https://docs.kde.org/trunk4/pt_BR/kdesdk/umbrello/uml-basics.html





FIM

Dúvidas?

CONTATO:
mileneserrano@unb.br
ou
mileneserrano@gmail.com

Sugestões?

UnB Gama